

CS330 - Computer Organization & Assembly Language

Assignment # 4

Individual Work Only

This assignment involves implementing bitwise manipulation algorithms in C. You will write C functions to perform fundamental operations such as checking, setting, clearing, and toggling specific bits, along with performing simple shift-based arithmetic. These exercises will deepen your understanding of binary representation and low-level data manipulation.

Important Note on Bit Positions

In this assignment, bit positions are counted from the right (least significant bit, LSB) starting at index 0.

Example:

Number 23 in binary → 0000000 ...0000 10111

Binary: 1 0 1 1 1

Index: 4 3 2 1 0

- Bit at position 0 = 1
- Bit at position 1 = 1
- Bit at position 2 = 1
- Bit at position 3 = 0
- Bit at position 4 = 1

Problem Description

Task 1: Check if a Bit is Set

Write a function `isBitSet` that takes an integer and a bit position, and returns 1 if the bit at that position is set, otherwise 0. **[20 points]**

```
int isBitSet(int number, int position) {  
  
    ...  
  
}
```

- Input: (23, 4)
- Output: 1 (since 23 → 10111, and bit at position 4 is set)

Task 2: Set a Bit

Write a function `setBit` that takes an integer and a bit position, and returns the new number with the bit at that position set to 1. **[20 points]**

```
int setBit(int number, int position) {  
    ...  
}
```

- Input: (5, 3)
- Output: 13 (since $5 \rightarrow 0101$, after setting bit 3 $\rightarrow 1101$)

Task 3: Clear a Bit

Write a function `clearBit` that takes an integer and a bit position, and returns the new number with the bit at that position cleared to 0. **[20 points]**

```
] int clearBit(int number, int position) {  
    ...  
}
```

- Input: (15, 1)
- Output: 13 (since $15 \rightarrow 1111$, after clearing bit 1 $\rightarrow 1101$)

Task 4: Toggle a Bit

Write a function `toggleBit` that takes an integer and a bit position, and flips the bit at that position. **[20 points]**

```
int toggleBit(int number, int position) {  
    ...  
}
```

- Input: (8, 3)
- Output: 0 (since $8 \rightarrow 1000$, after toggling bit 3 $\rightarrow 0000$)

Task 5: Multiply and Divide by 2 using Bit Shifts

Write a functions `multiplyBy2` and `divideBy2` that takes an integer and multiplies the number by 2, or divides the number by 2, respectively. **[each 10 points]**

```
int multiplyBy2(int number) {  
    ...  
}  
  
int divideBy2(int number) {  
    ...  
}
```

- Input: `multiplyBy2(7)` → Output: 14
 - Input: `divideBy2(18)` → Output: 9
-

Submission Requirements

- **Turn-in:**
 - `asgn2.h` **containing all functions** in a **header file**, which compiles and runs on Codespaces, and is documented appropriately
 - `asgn2.c` which contains the `main()` function, calls the other functions (in `asgn2.h`), compiles and runs on Codespaces, and is documented appropriately
 - a Makefile, where (a) the first rule compiles your code, and (b) there is a rule called “run” that will run the code. Therefore, the command “make” will compile the code, and the command “make run” will run the code. The template is provided on Canvas.
 - Independent completion form
 - All files should be inside a .zip file
- **Documentation:** Comment your code clearly to explain both *what* each part does and *why* you are implementing it that way. Avoid trivial or redundant comments.
 - Comments must be thorough and correct but not redundant.
 - Explain both what you’re doing and why you’re doing it.
 - Each function should be documented with (a) a short description of the function and what it does, (b) a description of each of the arguments, (c) a description of anything the function returns (or just state void).

- **Testing:** Include several test cases in your `main` function to demonstrate the correctness of each function.

Rules and Constraints

1. **No Standard Library Bitwise Helpers:** You must implement all logic using **bitwise operators** (`&`, `|`, `^`, `<<`, `>>`, `~`).
 1. You should not need to use any C library or syntax not discussed in lab. If you feel you need to use C syntax not discussed in lab, please coordinate with the TAs prior to submittal. **Using C syntax not discussed in lab may result in a zero on the assignment.**
2. **Error Handling:** Functions should handle invalid bit positions (e.g., negative positions or positions beyond the size of an integer) gracefully by returning the original number or an error value.
3. **Code Compilation:** Code that does not compile will result in an automatic zero.
4. **Bonus (20%):** Implement a function `countSetBits(int number)` that returns the number of 1s in the binary representation.
5. **NOTE: The following conditions will result in an automatic zero on the assignment.**
 1. **Code that does not compile on Codespaces by using the Makefile will result in an automatic zero.**
 2. **Submissions not in a .zip file**
 3. **Submissions missing a Makefile**
 4. **Submissions missing an Independent Completion form will result in an automatic zero.**
 5. **Using C syntax not discussed in lab.**
 6. **Submissions that utilize function names different from those stated above. For example, “isbitset” or “ISBITSET” instead of “isBitSet”**

Rubric

Criteria	Points
Working isBitSet function	20
Working setBit function	20
Working clearBit function	20
Working toggleBit function	20
Working multiplyBy2 function	10

Working divideBy2 function	10
Bonus: countSetBits	+20